

FastAPI

Revision Notes

Built from [app.rar](#) — Days 1 to 8

Sanjeev Thiyagarajan | freeCodeCamp FastAPI Course

github.com/abdullah090809/Working-With-FastAPI

Day	Topic	Key Concept
1	FastAPI Basics	Routes, Pydantic models, path/query params
2	CRUD (in-memory)	HTTP methods, status codes, HTTPException
3	Raw SQL (psycopg2)	Direct DB queries, <code>conn.commit()</code>
4	SQLAlchemy ORM	Models, sessions, ORM queries
5	Response models	<code>response_model</code> , <code>from_attributes</code> , filters output
6	Routers	<code>APIRouter</code> , <code>prefix</code> , <code>tags</code> , <code>include_router</code>
7	JWT + Hashing	<code>bcrypt</code> , <code>Create_Access_Token</code> , <code>/login</code>
8	Protected routes	<code>oauth2_scheme</code> , <code>Get_Current_User</code> , <code>Depends</code>

1. Project Structure

Every file in your `app/` folder and what it does.

```
app/
├── main.py           ← FastAPI app, registers routers, runs create_all
├── config.py         ← pydantic-settings: reads .env → Settings class
├── database.py       ← SQLAlchemy engine + SessionMaker + get_db()
├── oauth2.py         ← JWT: Create_Access_Token, Verify, Get_Current_User
├── utils.py          ← bcrypt: Hash(password), verify(plain, hashed)
├── models/
│   ├── posts.py     ← Post ORM model → "posts" table
│   └── users.py      ← User ORM model → "users" table
├── schemas/
│   ├── post.py       ← PostBase, PostCreate, PostUpdate, PostResponse
│   ├── user.py        ← UserCreate, UserResponse
│   └── token.py       ← Token (login response), TokenData (JWT payload)
└── routers/
    ├── post.py       ← CRUD routes for posts (all protected)
    ├── user.py        ← register + get user (no auth)
    └── auth.py        ← POST /login
```

Models vs Schemas — the key difference

	ORM Models (models/)	Pydantic Schemas (schemas/)
Inherits from	Base	BaseModel
Purpose	Represents a DB table	Validates request / filters response
Fields use	Column(...)	type annotations
Used by	SQLAlchemy for queries	FastAPI for validation & response_model

Why separate? User ORM model has a `password` column. `UserResponse` (schema) does not. So even when SQLAlchemy returns the full User object, `response_model=UserResponse` strips the password before it ever reaches the client.

2. Key Files — Your Actual Code

main.py

```
from fastapi import FastAPI
from app.database import Base, engine
from app.routers import post, user, auth

Base.metadata.create_all(bind=engine) # creates tables if missing

app = FastAPI()
app.include_router(post.router)
app.include_router(user.router)
app.include_router(auth.router)

@app.get("/")
def home():
    return "FastAPI Project"
```

- **create_all** — runs at startup, only creates tables that don't exist. Safe to leave in. To alter existing tables you need Alembic later.
- **include_router** — wires up routers. Each router defines its own prefix (/posts, /users) and tags for Swagger.
- `uvicorn app.main:app --reload` — start dev server. <http://127.0.0.1:8000/docs> opens Swagger.

config.py — Environment Variables

```
from pydantic_settings import BaseSettings

class Settings(BaseSettings):
    database_hostname: str
    database_port: str
    database_password: str
    database_name: str
    database_username: str
    secret_key: str
    algorithm: str
    access_token_expire_minutes: int

    class Config:
        env_file = ".env"

setting = Settings() # single instance, import this everywhere
```

- Reads from `.env` automatically at import time. If any variable is missing → validation error at startup.

- Import as: `from app.config import setting`
- **Never commit .env to git.** Add it to .gitignore.

database.py — SQLAlchemy Setup

```
SQLALCHEMY_DATABASE_URL = (
    f"postgresql://{setting.database_username}:{setting.database_password}"
    f"@{setting.database_hostname}:{setting.database_port}/{setting.database_name}"
)
engine = create_engine(SQLALCHEMY_DATABASE_URL)
SessionMaker = sessionmaker(autoflush=False, autocommit=False, bind=engine)
Base = declarative_base()

def get_db():
    db = SessionMaker()
    try:
        yield db          # gives session to the route function
    finally:
        db.close()       # always closes, even on exception
```

- **autocommit=False** — every write needs an explicit `db.commit()`. Forgetting this is a very common bug.
- **yield** — `get_db` is a generator. After the route finishes, execution returns here and finally runs `db.close()`.

models/posts.py — Post Table

```
class Post(Base):
    __tablename__ = "posts"
    id = Column(Integer, primary_key=True, nullable=False)
    title = Column(String, nullable=False)
    content = Column(String, nullable=False)
    published = Column(Boolean, server_default="TRUE", nullable=False)
    created_at = Column(TIMESTAMP(timezone=True),
                        nullable=False, server_default=text("now()"))
    owner_id = Column(Integer,
                      ForeignKey("users.id", ondelete="CASCADE"),
                      nullable=False)
    owner = relationship("User")
```

- **ondelete="CASCADE"** — at DB level, if a User is deleted all their Posts are deleted automatically.
- **relationship("User")** — not a DB column. When you access `post.owner`, SQLAlchemy auto-queries the users table. This enables the nested `owner: UserResponse` in `PostResponse`.
- **server_default** — the DB sets the value, not Python. Safer for timestamps.

oauth2.py — JWT Logic

```
oauth2_scheme = OAuth2PasswordBearer(tokenUrl="login")

def Create_Access_Token(data: dict):
    payload = data.copy()
    expire = datetime.utcnow() + timedelta(minutes=ACCESS_TOKEN_EXPIRE_MINUTES)
    payload.update({"exp": expire})
    return jwt.encode(payload, SECRET_KEY, algorithm=ALGORITHM)

def Verify_Access_Token(token: str, credentials_exception):
    try:
        payload = jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
        id = payload.get("id")
        if id is None: raise credentials_exception
        token_data = TokenData(**payload) # ignores "exp" field
        return token_data
    except JWTError:
        raise credentials_exception

def Get_Current_User(token=Depends(oauth2_scheme), db=Depends(get_db)):
    credentials_exception = HTTPException(
        status_code=401, detail="Could not validate Credentials",
        headers={"WWW-Authenticate": "Bearer"})
    token = Verify_Access_Token(token, credentials_exception)
    user = db.query(User).filter(User.id == token.id).first()
    return user # ← add: if not user: raise credentials_exception
```

Known gap: After fetching the user there is no None check. If a user is deleted after their token was issued, `Get_Current_User` returns None and protected routes crash. Fix: add `if not user: raise credentials_exception` after the query.

utils.py — Password Hashing

```
pwd_context = CryptContext(schemes=["bcrypt"], deprecated="auto")

def Hash(password: str):
    return pwd_context.hash(password) # bcrypt hash — stored in DB

def verify(plain_password, hashed_password):
    return pwd_context.verify(plain_password, hashed_password) # True/False
```

- You never decrypt a bcrypt hash — comparison only.
- Hash called in routers/user.py on register. verify called in routers/auth.py on login.

3. Schemas (Pydantic)

Schema Inheritance Pattern

```
PostBase(BaseModel)          # title, content, published
├─ PostCreate(PostBase)      # used for POST body – no changes
└─ PostResponse(PostBase)    # adds id, created_at, owner_id, owner

PostUpdate(BaseModel)        # all Optional – used for PATCH body

UserBase(BaseModel)          # email (EmailStr), password
└─ UserCreate(UserBase)      # used for POST /users body

UserResponse(BaseModel)      # id, email, created_at – NO password
```

PostResponse — the full schema

```
class PostResponse(PostBase):
    id: int
    created_at: datetime
    owner_id: int
    owner: UserResponse      # nested – serializes the related User
    class Config:
        from_attributes = True # Pydantic v2 (was orm_mode=True in v1)
```

from_attributes = True — SQLAlchemy objects are not dicts. This tells Pydantic to use `getattr(obj, "field")` instead of `obj["field"]`, so it can read ORM model instances.

Pydantic v1 vs v2 — Changes in Your Code

v1 (old – do not use)	v2 (what you use)
<code>orm_mode = True</code>	<code>from_attributes = True</code> (in class Config)
<code>post.dict()</code>	<code>post.model_dump()</code>
<code>post.dict(exclude_unset=True)</code>	<code>post.model_dump(exclude_unset=True)</code>

PostUpdate — all Optional for PATCH

```
class PostUpdate(BaseModel):
    title: Optional[str] = None
    content: Optional[str] = None
```

```
published: Optional[bool] = None

# In the route:
post_query.update(updated_post.model_dump(exclude_unset=True), ...)
#
# Without exclude_unset=True: {"title": "x", "content": None} → overwrites!
# With    exclude_unset=True: {"title": "x"}                → safe
```

4. All Routes

Posts — prefix: /posts — all require auth

Method	Path	Description	Notes
GET	/posts/	Get all posts	?limit=10&skip=0&search=
GET	/posts/{id}	Get one post	404 if not found
POST	/posts/	Create post	owner_id = current_user.id 201
PUT	/posts/{id}	Full update	403 if not owner of post
PATCH	/posts/{id}	Partial update	exclude_unset=True 403 if not owner
DELETE	/posts/{id}	Delete post	204 no content 403 if not owner

Users — prefix: /users — no auth required

Method	Path	Description	Notes
POST	/users/	Register user	Hashes password before save 201
GET	/users/{id}	Get user by id	404 if not found

Auth — no prefix

Method	Path	Description	Notes
POST	/login	Login	form-data (not JSON) returns JWT

Status Codes

Code	Meaning	When used in your API
200	OK	Default for GET, PUT, PATCH
201	Created	POST /posts/ and POST /users/
204	No Content	DELETE — success with no response body
401	Unauthorized	Bad/missing/expired token — raised in Get_Current_User
403	Forbidden	Wrong password at login, or editing someone else's post
404	Not Found	Post or user not found
422	Unprocessable Entity	Pydantic validation failed — auto-raised by FastAPI

5. Auth Flow — End to End

Step 1 — Register

```
POST /users
→ UserCreate body (email, password)
→ Hash(password)           # plain text never saved
→ User(**user.model_dump()) # ORM object
→ db.add() → db.commit() → db.refresh()
→ return UserResponse      # no password field
```

Step 2 — Login

```
POST /login
→ OAuth2PasswordRequestForm # form-data, NOT JSON
→ user_credentials.username # the "username" field = your email
→ db.query(User).filter(User.email == user_credentials.username)
→ verify(plain, hashed)     # bcrypt comparison
→ Create_Access_Token({"id": user.id}) # JWT with 30min expiry
→ return {"access_token": token, "token_type": "bearer"}
```

Important: OAuth2PasswordRequestForm reads **form-data**, not JSON.

In Postman: Body → x-www-form-urlencoded. Field name is `username` (not email) — OAuth2 spec requirement, even though you pass an email value.

Both "user not found" and "wrong password" return 403 "Invalid Credentials" — intentionally vague so attackers can't tell which failed.

Step 3 — Protected Request

```
GET /posts (or any protected route)
→ Header: Authorization: Bearer <token>
→ oauth2_scheme extracts token string
→ Depends(Get_Current_User) fires:
    jwt.decode(token, SECRET_KEY, algorithms=[ALGORITHM])
    payload.get("id") → TokenData(id=5)
    db.query(User).filter(User.id == token.id).first()
    return user # injected as current_user in route
→ If token invalid/missing/expired → 401 Unauthorized
```

Inside the JWT

```
# What you encode:
{"id": user.id, "exp": datetime.utcnow() + timedelta(minutes=30)}
```

```
# jose signs it → base64url string: header.payload.signature
# The payload is NOT encrypted – anyone can decode it
# But they cannot forge a valid signature without SECRET_KEY
# → Never put passwords or sensitive data in the JWT payload
```

6. Patterns & Gotchas

The Query Object Pattern — most important pattern in your code

```
post_query = db.query(Post).filter(Post.id == id) # Query — no DB hit
post       = post_query.first()                 # Result — hits DB now

# Use post (result) for:      reading values, 404 check, ownership check
# Use post_query (query) for: .update(), .delete()

# WRONG — calling .update() on the result object crashes
post.update(...)             # AttributeError

# CORRECT
post_query.update(updated_post.model_dump(), synchronize_session=False)
db.commit()
```

Why two variables? `.update()` and `.delete()` must be called on the Query object, not the fetched result. But you need the result to read `post.owner_id` for ownership checks. Store both every time.

`db.refresh()` — always after commit on a new object

```
db.add(new_post)
db.commit()
db.refresh(new_post) # re-fetches row from DB
return new_post      # now has id, created_at, server defaults

# Without refresh: new_post.id = None, new_post.created_at = None
# The commit saved to DB, but the Python object is still stale
```

`Depends()` — dependency injection

```
@router.post("/", response_model=PostResponse)
def create_post(
    post: PostCreate,                # request body
    db: Session = Depends(get_db),   # DB session injected
    current_user: User = Depends(Get_Current_User) # auth user injected
):
    # FastAPI calls get_db() and Get_Current_User() automatically
    # You never call them manually
    new_post = Post(owner_id=current_user.id, **post.model_dump())
    ...
```

response_model filters output

```
@router.post("/users", response_model=UserResponse) # UserResponse has no
password
def create_user(...):
    return new_user # User ORM object - HAS .password attribute
                  # password is stripped by response_model before sending
```

PUT vs PATCH

	PUT	PATCH
Schema used	PostCreate (all required)	PostUpdate (all Optional)
model_dump call	post.model_dump()	post.model_dump(exclude_unset=True)
If client sends only title	content and published overwritten with None	Only title is updated

401 vs 403

Code	Meaning	When you use it
401 Unauthorized	Authentication problem	No token, bad token, expired — raised in Get_Current_User
403 Forbidden	Authenticated but not allowed	Wrong password at login, or editing someone else's post

7. Quick Quiz — Test Yourself

Cover the answers and try to answer from memory. Uncover to check.

1. After `db.query(Post).filter(...)`, why do you store the query before calling `.first()`?

Answer: Because `.update()` and `.delete()` must be called on the Query object, not the fetched result. You need the result (`post`) for reading values and ownership checks, and the query (`post_query`) for mutations.

2. `OAuth2PasswordRequestForm` expects what format? What is the field called?

Answer: `form-data` (`x-www-form-urlencoded`), not JSON. The field is called "username" even when you pass an email — that is what the OAuth2 spec requires.

3. What does `model_dump(exclude_unset=True)` do? When do you use it?

Answer: Only includes fields the client explicitly sent. Used on PATCH routes so unset fields do not overwrite existing DB values with None.

4. Why call `db.refresh(new_post)` after `db.commit()`?

Answer: The Python object is stale after commit — it does not automatically receive DB-generated values like `id`, `created_at`, or server defaults. `refresh()` re-fetches the full row.

5. How does `owner: UserResponse` work in `PostResponse` without a JOIN in the route?

Answer: The Post ORM model has `owner = relationship("User")`. SQLAlchemy auto-fetches the related User when `post.owner` is accessed. Pydantic serializes it using `UserResponse`, which has no password field.

6. When do you raise 401 vs 403?

Answer: 401 = authentication problem (bad/missing/expired token) — raised in `Get_Current_User`. 403 = authenticated but not allowed (wrong password at login, or editing someone else's post).

7. What does `from_attributes = True` do in a schema Config?

Answer: Tells Pydantic to use `getattr(obj, "field")` instead of `obj["field"]`, so it can read SQLAlchemy ORM model instances rather than just dicts.

8. Re-entry Checklist

Do these in order before writing any new code after a break.

Environment

- PostgreSQL service is running
- `.env` file exists in project root with all 8 variables
- Virtual environment is activated
- All packages installed (`pip install -r requirements.txt`)

Startup

- Run `uvicorn app.main:app --reload` — no errors in terminal
- Open `http://127.0.0.1:8000/docs` — three sections: Posts, Users, Authentication
- Tables exist in DB (`create_all` runs at startup — check pgAdmin)

Smoke Test

- POST `/users` — register a test user via Swagger
- POST `/login` — use `x-www-form-urlencoded`, get back a token
- Click "Authorize" in Swagger, paste the token
- POST `/posts` — create a post, confirm it returns owner nested inside
- DELETE `/posts/{id}` as a different user — confirm 403

9. What's Next — Hours 10 to 19

Votes / Likes (many-to-many)

```
class Vote(Base):
    __tablename__ = "votes"
    post_id = Column(Integer, ForeignKey("posts.id", ondelete="CASCADE"),
                     primary_key=True)
    user_id = Column(Integer, ForeignKey("users.id", ondelete="CASCADE"),
                     primary_key=True)
    # Composite primary key — one user cannot vote on same post twice
```

SQLAlchemy Joins — post + vote count

```
from sqlalchemy import func

posts = db.query(Post, func.count(Vote.post_id).label("votes")) \
    .outerjoin(Vote, Vote.post_id == Post.id) \
    .group_by(Post.id).all()
```

Alembic — proper migrations

```
# create_all() cannot modify existing tables
# Alembic generates migration scripts like Django migrations

alembic init alembic
alembic revision --autogenerate -m "add votes table"
alembic upgrade head
```

CORS — allow frontend to call API

```
from fastapi.middleware.cors import CORSMiddleware
app.add_middleware(CORSMiddleware,
                   allow_origins=["*"], # or specific domains in production
                   allow_methods=["*"],
                   allow_headers=["*"])
```

Testing — pytest + TestClient

```
from fastapi.testclient import TestClient
client = TestClient(app)
```

```
def test_create_post():
    res = client.post("/posts", json={"title": "test", "content": "hello"})
    assert res.status_code == 201
```

Deployment Stack

Layer	Tool	Purpose
App server	Gunicorn + uvicorn workers	Multi-process ASGI serving
Reverse proxy	Nginx	SSL termination, static files, load balance
SSL	Let's Encrypt (certbot)	Free HTTPS certificate
OS	Ubuntu 22.04 LTS	Cheap VPS (DigitalOcean, AWS EC2)

You are roughly halfway through. The hard conceptual parts (ORM, auth, JWT) are done. The second half is mostly production concerns — migrations, testing, deployment. The code does not get much harder, just more structured.